

Self-Balancing Robot

Nonlinear TWIP dynamics (RK4, 1 kHz), IMU fusion (CF vs Kalman), cascaded PID, stability analysis

AT A GLANCE

OK

Balance from 10 deg

0.33 / 0.59 deg

Tilt RMSE: KF+enc / CF

rise 0.96 s, OS 12 %

Velocity step 0.4 m/s

7.6 deg, back in 0.76 s

Push 0.4 N s: max tilt

+10.6 1/s

Open-loop unstable pole

64/100

Gain sweep stable cells

18/18

Unit tests

VERDICT

SIM PASS

TESTS 18/18

CL POLES STABLE

DECLARED SUBSTITUTION

Eswar Mahalingam
Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

1. Objective and brief mapping

I modelled a two-wheeled inverted-pendulum robot (1.2 kg body, 0.12 m COM height, 40 mm wheels, two 0.35 N m motors with a torque-speed line and a 10 ms lag, viscous friction) as coupled nonlinear Lagrange equations integrated with RK4 at 1 kHz, simulated an MPU-6050-class IMU (gyro bias + random walk + white noise, accelerometer with lever-arm effects, bias and noise) and quantised wheel encoders, and closed the loop with a cascaded PID (1 kHz tilt loop, 100 Hz velocity loop, yaw-rate loop) fed by either a complementary filter or a 2-state Kalman filter. Scripted forward/back/turn commands and a keyboard-teleop stub drive the robot; the stability analysis linearises the model numerically and computes open- and closed-loop poles, step metrics, disturbance rejection and a Kp x Kd sweep.

Every number here comes from `sim/run_sim.py` and the unittest run in this container (`sim/metrics.json`, `sim/track_log.csv`, `sim/test_output.log`).

2. Architecture

`src/balancebot/` is shared by the simulator, the tests and the ROS 2 node; the Gazebo SDF carries the same masses, inertias, torque and IMU noise figures.

commands (CommandStream / KeyboardTeleopStub / ROS cmd_vel) ---> v_ref, yaw_rate_ref

```

TWIP.f(s,u) <-- RK4 1 kHz -- tau_L, tau_R <-- CascadedController: vel PID (100 Hz) -> tilt_ref
|
|
+--> IMU.measure: gyro (bias+noise), acc x/z (lever arm, noise) --+-- TiltKalman / Complementary
+--> Encoder: windowed counts -> v_meas, a_est -> tilt_from_acc_compensated(acc, a_est, thet
analysis.py: linearise (central differences) -> A (8x8), reduce to [x, xd, th, thd, tau] -> pole

```

3. Implementation details

- **Dynamics:** state $[x, xd, th, thd, psi, psid, tau_L, tau_R]$; 2×2 mass matrix solved every call; motor torque limited to $tau_stall (1 - |w|/w_noload)$ and lagged by $T = 10$ ms; yaw from differential torque. RK4 energy drift on the friction-free free pendulum is below $1e-6$ over 0.4 s (unit test).
- **IMU:** gyro = $th_dot + bias$ (1.5 deg/s initial, 0.002 deg/s per sample random walk) + $N(0, 0.3 \text{ deg/s})$; accelerometer = specific force of a point 0.10 m up the body (includes $h*th_ddot$ and $h*th_dot^2$ terms) rotated into the body frame + bias (0.03, -0.02 m/s²) + $N(0, 0.15 \text{ m/s}^2)$. Tilt from the accelerometer alone: $atan2(-f_x, f_z)$.
- **Estimators:** complementary filter $\tau = 0.5$ s; Kalman filter $x = [theta, gyro_bias]$, $F = [[1, -dt], [0, 1]]$, $H = [1, 0]$, $Q = \text{diag}(1e-6, 1e-7)$, $R = 0.1 \text{ rad}^2$, R inflated by $1 + 200 (|f| - g)^2 / g^2$ while the robot accelerates, angle prior 1 deg and bias prior 0.5 deg/s. Both are also run on an accelerometer tilt that has the encoder-derived body acceleration removed (a_est from 20 ms count windows, 20 ms low-pass).
- **Controller:** tilt PID Kp 9.0, Ki 4.0, Kd 0.9 (N m per rad, rad s, rad/s) with conditional-integration anti-windup, integral clamp 0.3, derivative on the bias-corrected gyro through a 5 ms low-pass; velocity PI Kp 0.2 rad per m/s, Ki 0.08, tilt-reference clamp ± 8 deg, run at 100 Hz; yaw-rate PI Kp 0.06, Ki 0.02 N m per rad/s. Total torque clamp 0.7 N m.
- **Why Kd went from 0.45 to 0.9:** the first tuning had fast closed-loop poles at $-9.8 \pm 51j$ ($\zeta = 0.19$). With ideal state feedback it worked, but once the encoder window and derivative filter added a few milliseconds of lag the robot fell into a 2.5 Hz limit cycle with velocity swings of ± 0.45 m/s. Kd = 0.9 moves the fast poles to $-42.8 \pm 116.8j$ ($\zeta = 0.34$) and the limit cycle disappears at every outer-loop gain I tried.

4. Verification

`python3 -m unittest discover -s tests -v: 18 tests`, all passing. The closed-loop tests call `sim/run_sim.simulate()` directly, so the tests, the figures and this report share one code path; the whole suite runs in about 3.5 s because the model is small enough for 1 kHz RK4 in pure numpy.

Test group	What is checked	Result
Dynamics (6)	upright is an equilibrium, falls open-loop from 1 deg, open-loop pole real and > 5 1/s, torque sign convention, motor saturation + lag follow the torque-speed line, RK4 energy conservation $< 1e-6$	pass
Sensors + filters (5)	static accelerometer tilt exact, KF learns the gyro bias to 0.3 deg/s and the angle to 0.3 deg at rest, CF steady error = bias x tau, encoder velocity to 1 %, PID anti-windup clamp	pass
Closed loop (7)	balances from 10 deg ($ th < 1$ deg, $ v < 0.1$ m/s at 4 s), velocity step rise < 2 s and overshoot < 40 %, yaw rate tracks 1 rad/s to 0.05, recovers from a 0.4 N s push, KF+enc RMSE < 0.5 deg and CF+enc < 1 deg, all closed-loop poles in the left half-plane (one free position integrator), keyboard stub mapping	pass

5. Measured performance

The velocity response is non-minimum-phase, as it must be for this robot: to speed up, the wheels first roll backwards to tip the body forward, so v dips before it rises and the 10-90 % rise time is measured from the reference step. The impulse test pushes the body backwards by 7.6 deg and the robot rolls 6.5 cm to catch it. The 0.4 m/s tracking RMSE is dominated by the four reference steps (the reference is a square wave the robot cannot follow instantly); once settled the velocity ripple is about ± 0.01 m/s, which is the encoder quantisation acting through the outer loop.

KEY TAKEAWAY The tuned cascade holds the robot from 10 deg, tracks 0.4 m/s with 12 % overshoot and survives a 0.4 N s push; a Kalman filter fed with encoder-compensated accelerometer tilt cuts the estimation RMSE to 0.33 deg. Gazebo SDF + ROS 2 provided, not run here.

04

Robotics & Automation
Codec Technologies India
Internship 2026

Self-Balancing Robot

Nonlinear TWIP dynamics (RK4, 1 kHz), IMU fusion (CF vs Kalman), cascaded PID, stability analysis

AT A GLANCE

OK

Balance from 10 deg

0.33 / 0.59 deg

Tilt RMSE: KF+enc / CF

rise 0.96 s, OS 12 %

Velocity step 0.4 m/s

7.6 deg, back in 0.76 s

Push 0.4 N s: max tilt

+10.6 1/s

Open-loop unstable pole

64/100

Gain sweep stable cells

18/18

Unit tests

VERDICT

SIM PASS

TESTS 18/18

CL POLES STABLE

DECLARED SUBSTITUTION

Eswar Mahalingam

Robotics & Automation Intern

Ghaziabad, UP, India

+91-9360548243

eswarmba05313@gmail.com

linkedin.com/in/eswar-mahalingam

Experiment	Metric	Value
Balance from 10 deg	settled within 1 deg after / peak reverse tilt / max torque / ms at torque limit	0.647 s / -1.43 deg / 0.7 N m / 18 ms
Velocity step 0 -> 0.4 m/s	rise (10-90 %) / overshoot / settling (10 % band) / steady error	0.956 s / 12.294 % / 3.949 s / 0.033 m/s
Yaw-rate step 0 -> 1 rad/s	rise / overshoot / settling	0.035 s / 0.624 % / 0.042 s
18 s command stream	velocity RMSE vs reference (incl. transients) / max tilt / distance	0.1115 m/s / 5.65 deg / 0.538 m
Push 8 N x 50 ms (0.4 N s)	max tilt / back within 1 deg after / displacement / peak torque	7.64 deg / 0.764 s / 0.065 m / 0.181 N m
Estimator RMSE (3 seeds, 12 s manoeuvre)	accel only / CF raw / CF+enc / KF raw / KF+enc	2.742 / 1.342 / 0.586 / 1.737 / 0.33 deg
Open-loop poles (planar, upright)	eig(A) of [x, xd, th, thd]	0.00, -0.03, -10.75, 10.55
Closed-loop poles, tilt loop only	with motor lag + tilt integrator	0.00, -45.15 ± 146.75j, -9.43, -0.49, -0.01
Closed-loop poles, full cascade	7 states incl. both integrators	0.00, -42.85 ± 116.83j, -12.30, -1.24, -0.50 ± 0.10j
Kp x Kd sweep (10 x 10, 3 s from 10 deg)	stable cells / best settling / at (Kp, Kd)	64/100 / 0.198 s / (12.0, 1.0)

5b. Linearised model and gain sweep

Settling time (s, |theta| < 1 deg for the rest of a 3 s run from 10 deg) on a subset of the 10 x 10 Kp x Kd sweep. Low Kd falls at any Kp above ~6 N m/rad (too little damping for the motor lag); high Kp with moderate Kd is fastest but leaves less margin for sensor lag, which is why the chosen point sits in the middle of the stable region rather than at the fastest cell.

$$A = \begin{bmatrix} 0.000 & 1.000 & 0.000 & 0.000 & 0.000 \\ 0.000 & -0.070 & -10.887 & 0.015 & 42.871 \\ 0.000 & 0.000 & 0.000 & 1.000 & 0.000 \\ 0.000 & 0.385 & 113.410 & -0.161 & -272.961 \\ 0.000 & 0.000 & 0.000 & 0.000 & -100.000 \end{bmatrix}$$

$$B^T = \begin{bmatrix} 0.000 & 0.000 & 0.000 & 0.000 & 100.000 \end{bmatrix}$$

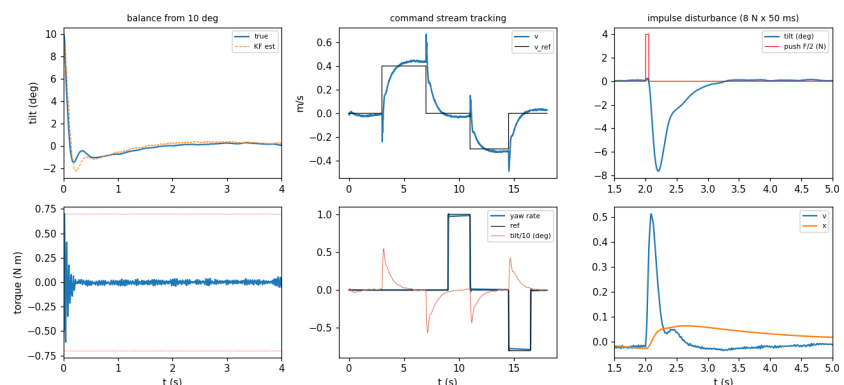
Kd \ Kp	2	6	10	14	18	20
0.16	3.00	fell	fell	fell	fell	fell
0.37	1.37	3.00	3.00	fell	fell	fell
0.58	2.65	0.66	2.23	3.00	fell	fell
0.79	3.00	0.85	0.51	1.74	3.00	3.00
1.00	3.00	0.91	0.60	0.74	1.46	2.99

5c. Command-stream segments (sim/track_log.csv)

Each row is one setpoint change of the scripted stream; the response columns are read from the logged trajectory. The turn commands are followed within 50 ms because yaw is a first-order loop on the differential torque, while every velocity change costs a 4-6 deg lean.

t (s)	v_ref (m/s)	yaw_ref (rad/s)	v after 1.5 s	yaw rate after 0.5 s	max tilt in segment (deg)
3.0	+0.4	+0.0	+0.397	+0.000	5.5
7.0	+0.0	+0.0	+0.007	+0.000	5.6
9.0	+0.0	+1.0	-0.029	+0.975	0.2
11.0	-0.3	+0.0	-0.305	+0.012	4.3
14.5	+0.0	-0.8	+0.004	-0.776	4.2
16.5	+0.0	+0.0	+0.030	-0.007	0.2

6. Figures



Top: tilt (true vs KF estimate) from 10 deg, velocity tracking of the command stream, tilt during the push. Bottom: torque with the 0.7 N m limits, yaw-rate tracking with tilt/10 overlaid, velocity and position after the push.

KEY TAKEAWAY The tuned cascade holds the robot from 10 deg, tracks 0.4 m/s with 12 % overshoot and survives a 0.4 N s push; a Kalman filter fed with encoder-compensated accelerometer tilt cuts the estimation RMSE to 0.33 deg. Gazebo SDF + ROS 2 provided, not run here.

04

Robotics & Automation
Codec Technologies India
Internship 2026

Self-Balancing Robot

Nonlinear TWIP dynamics (RK4, 1 kHz), IMU fusion (CF vs Kalman), cascaded PID, stability analysis

AT A GLANCE

OK

Balance from 10 deg

0.33 / 0.59 deg

Tilt RMSE: KF+enc / CF

rise 0.96 s, OS 12 %

Velocity step 0.4 m/s

7.6 deg, back in 0.76 s

Push 0.4 N s: max tilt

+10.6 1/s

Open-loop unstable pole

64/100

Gain sweep stable cells

18/18

Unit tests

VERDICT

SIM PASS

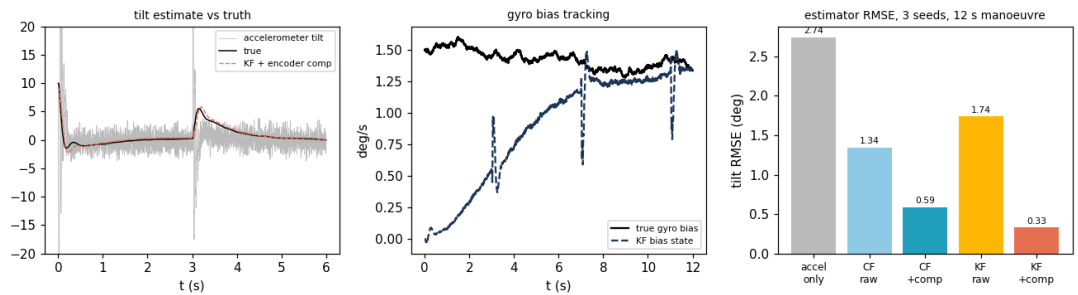
TESTS 18/18

CL POLES STABLE

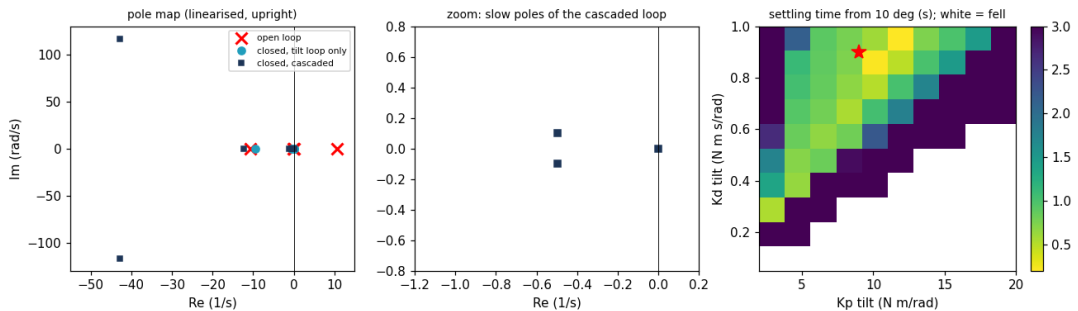
DECLARED SUBSTITUTION

Eswar Mahalingam

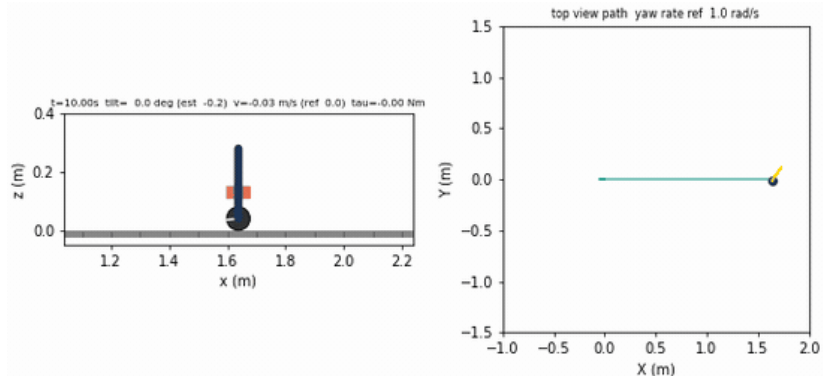
Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam



Left: accelerometer-only tilt (grey), truth (black) and KF+encoder estimate (orange). Middle: the KF bias state following the gyro bias random walk. Right: RMSE of the five estimators over three seeds.



Pole map of the linearised model (open loop has a pole at +10.6 1/s), zoom on the slow cascaded poles, and the Kp x Kd settling-time heat-map (white = fell; star = chosen gains).



GIF frame during the 1 rad/s turn: side view with wheel spoke and body, and the top-view path built from the yaw integration.

6b. Keyboard teleop stub (sim/metrics.json: keyboard_teleop_demo)

w/s step the velocity setpoint by 0.3 m/s (clamped to 0.6), a/d step the yaw-rate setpoint by 0.8 rad/s, space or x stops. The stub produces the same (v_ref, yaw_rate_ref) pair that CommandStream and the ROS/cmd_vel subscriber feed to CascadedController.set_command().

Key sequence	'w'	'w'	'a'	'd'	's'	's'	' '	'x'
v_ref (m/s)	+0.3	+0.6	+0.6	+0.6	+0.3	+0.0	+0.0	+0.0
yaw_ref (rad/s)	+0.0	+0.0	+0.8	+0.0	+0.0	+0.0	+0.0	+0.0

7. Declared substitutions / not run here

- **Gazebo:** worlds/balance_bot.sdf (body + two wheels with the same masses, inertias, effort/velocity limits, IMU sensor with the same noise figures, gazebo_ros IMU and joint-state plugins). Gazebo is not installed in the build container; the file was written by hand and validated only as XML.
- **ROS 2 Humble:** ros2_ws/src/balance_bot/ with node.py (subscribes /imu, /joint_states, /cmd_vel; publishes /wheel_torque, /tilt_estimate, /odom) plus package.xml, setup.py, setup.cfg, resource marker and launch file with the tuned gains as parameters. py_compile-checked only.
- **Keyboard teleop:** KeyboardTeleopStub maps w/s/a/d/space to setpoints; the actual key capture (curses / teleop_twist_keyboard) needs a terminal and is not exercised here.

8. Learnings / next steps

Linear pole placement was necessary but not sufficient: the design that looked fine on the pole map limit-cycled as soon as realistic sensing lag was in the loop, and the fix (more damping) was only obvious after isolating perfect-state feedback from the sensor chain. The other lesson is that a Kalman filter whose Q and R come straight from the datasheet is worse than a complementary filter here, because the accelerometer error is the robot's own motion, not white noise; compensating it with the encoders is what made the KF win. Next: LQR on the linearised model as a comparison, a friction/backlash model for cheap gear motors, and running the SDF in Gazebo with the node.

KEY TAKEAWAY The tuned cascade holds the robot from 10 deg, tracks 0.4 m/s with 12 % overshoot and survives a 0.4 N s push; a Kalman filter fed with encoder-compensated accelerometer tilt cuts the estimation RMSE to 0.33 deg. Gazebo SDF + ROS 2 provided, not run here.